Reporte de preparación para la presentación

WikiLLM frente a RAG sobre un corpus de arXiv

Maestría en Inteligencia Artificial Aplicada · Juan David Cruz y Juan Diego Ramírez

En una frase

La demostración funciona de punta a punta en local. Con el **mismo modelo** (`deepseek-flash`), el **RAG** y el **WikiLLM** se abstienen cuando el corpus no tiene la respuesta y, tras ingerir el paper que la contiene, responden lo mismo y citan el mismo fragmento. En el piloto, el WikiLLM acierta más (5 de 6 frente a 2 de 6), pero lee unas 2,9 veces más contexto y cuesta más. El piloto **no** es un resultado del informe.

Índice

1. El proyecto en una página 1

2. Estado de la demostración (verificado hoy) 2

2.1. Qué se arregló hoy . . . 2

2.2. Mejoras del panel para la presentación . . . 3

3. Guion de la demostración 3

4. Resultados del piloto y cómo contarlos 4

5. Preguntas probables del jurado 5

6. Plan B y cierre 6

1. El proyecto en una página

Problema. Un RAG recupera en cada consulta los fragmentos más parecidos a la pregunta, sin distinguir una fuente reciente de una superada, y repite la síntesis en cada respuesta. Un wiki mantenido por un LLM traslada ese trabajo a la ingesta: integra cada fuente nueva en páginas existentes y responde consultando conocimiento ya organizado, a cambio de un costo de mantenimiento. La literatura que compara ambos enfoques usa corpus, modelos y métricas distintos y no converge.

Pregunta de investigación.

> *¿Cómo se comparan un sistema RAG simple y un WikiLLM, con el mismo modelo, a medida que se ingiere un corpus de forma cronológica, en correctitud, fundamentación, costo y tiempo?*

Hipótesis de trabajo. La recuperación por similitud tiende a devolver el fragmento más parecido aunque esté superado; un wiki mantenido incrementalmente puede reflejar la información más reciente. La diferencia debería verse en las preguntas T4 (actualización). La contrapartida esperada es un costo mayor en el wiki. La formulación es abierta: un resultado desfavorable al WikiLLM también es válido y se reporta.

Diseño, en cuatro ideas.

1. **Una sola variable.** Mismo modelo, mismo corpus, mismas preguntas, misma frase de abstención y misma extracción de citas. Solo cambia la arquitectura de la memoria.
2. **Corpus de 50 preprints** de arXiv (casi todos de 2026, posteriores al corte del modelo) procesados en Bronze → Silver → Gold. Cada párrafo, tabla o ecuación tiene un `span_id` con su huella: una cita solo vale si resuelve a uno real.
3. **Ingesta cronológica.** Los papers entran de uno en uno; tras los pasos 0, 5, 10, …, 50 se hacen las mismas preguntas a los dos sistemas.
4. **Respuestas fijadas por humanos.** Banco de 60 preguntas (T1 hecho puntual, T2 síntesis, T3 comparación, T4 actualización, T5 no respondible). Una pregunta solo cuenta cuando los dos autores la verificaron contra el paper.

Los dos sistemas.

	RAG simple	WikiLLM
Memoria	Vectores (Chroma, `bge-small-en-v1.5` en GPU), 5.430 fragmentos	Carpeta de páginas Markdown con índice, registro y Git propio
Al ingerir	Nada: el índice ya está hecho	2 llamadas (plan y escritura) y validación por código de citas, enlaces y cifras
Al responder	Top-5 por similitud y **1 llamada**	Un agente que **busca y lee** páginas (tope de 8 acciones)

2. Estado de la demostración (verificado hoy)

Componente	**Estado**
Entorno (`uv`, torch con CUDA)	Listo; torch 2.14.1+cu126 detecta la RTX 3050
Índice del RAG (`data/chroma`)	Reconstruido: 50 papers, 5.430 fragmentos, 72 s en GPU
Wiki base (`data/wiki/dev2`)	Reconstruido: 5 papers, 18 páginas, 0,043 USD
Wiki de la demo (`data/wiki/demo`)	En el paso 5, etiquetas `paso-1` a `paso-5`
`scripts/verificar_demo.py`	10 de 11 (la falla restante: cambios sin commit)
Calidad del código	`ruff` sin problemas; **449 pruebas** pasan
Panel web	`http://127.0.0.1:8765`

2.1. Qué se arregló hoy

1. **Error de código al preguntar.** Torch fallaba con `libcufft.so.11: version not found`: la caché de `uv` tenía paquetes de NVIDIA dañados por una instalación interrumpida. Se limpiaron y se reinstalaron.
2. **Faltaban datos que no se versionan**: el índice del RAG y el wiki base. Se reconstruyeron.
3. **Resultados se quedaba en «Cargando…».** El frontend cargaba las métricas pero repintaba otra zona de la página. Corregido.
4. **Métricas del piloto desactualizadas.** Tras la prueba de reanudación se rehicieron 10 respuestas (5 preguntas T5), pero las métricas versionadas seguían calculadas con la primera tanda. Se regeneraron con la capa determinista (sin juez, sin gasto); el cálculo es idempotente. Registrado en `docs/bitacora-datos.md`.
5. **Verificación del Gold.** Exigía exactamente 50 archivos y fallaba con dos papers fuera del corpus (los de las T5). Ahora comprueba que esté el Gold de cada paper del manifest.

6. **Comando de reinicio en Linux.** La consola mostraba la orden de PowerShell; ahora muestra la de cada sistema operativo (con prueba).

2.2. Mejoras del panel para la presentación

- **Barra «Recorrido»**: los 6 pasos del guion; resalta el que sigue y precarga la pregunta de BeliefShift.
- **«Lectura rápida»** en cada comparación, en lenguaje llano: «Los dos se abstuvieron…», «citan el mismo fragmento…», «El RAG usó 2,2× menos tokens de entrada…».
- **Grafo del wiki** (tipo Obsidian) en la sección Wiki: nodos por página (papers en índigo, conceptos en verde) unidos por los enlaces `[[...]]`; resalta los vecinos al pasar el cursor y marca con un anillo las páginas que tocó la última ingesta. Funciona sin internet.
- **Resultados**: tabla RAG frente a WikiLLM con el mejor valor de cada métrica resaltado, indicando si conviene que sea alto o bajo, y un resumen en una frase.

3. Guion de la demostración

Duración: 10–12 minutos. Costo en vivo: menos de 0,02 USD (tope de la sesión: 0,25 USD). Las cifras vienen del ensayo real de hoy; la redacción y los centavos pueden variar un poco porque el proveedor no es determinista.

Antes de empezar (15 minutos antes)

1. `uv run python -m wikillm.panel` y esperar el índice del RAG (unos 15 s).
2. **Sistema** → **Reiniciar el wiki**: la barra debe decir «5 de 50 papers».
3. **Sistema** → **Verificar**: 11 de 11 (o 10 si hay cambios sin commit).
4. Revisar la hora: **evitar 20–23 h y 01–05 h de Colombia** (hora pico de DeepSeek, tarifa doble).

Paso	Qué se hace	Qué se espera ver	USD
1	Wiki en el paso 5	Grafo con 18 páginas y 39 enlaces; índice, una página con `[span_id]` tras cada afirmación, `log.md` y etiquetas `paso-1`…`paso-5`	0
2	Preguntar antes: *What is the total number of sessions in the BeliefShift dataset?*	Los dos se abstienen. El RAG recupera fragmentos de otros papers (distancia ≈ 0,33); el agente busca 3 veces con palabras distintas y no encuentra páginas	0,0004
3	Ingerir el paper 6 (BeliefShift)	≈ 40 s, 2 llamadas: plan, escritura, validación y copia `paso-6`. Ensayo: 3 páginas creadas, 5 actualizadas, 0 citas eliminadas	0,009
4	Preguntar después	Los dos responden **68,160** y citan `2603.23848#s18.p3`. El agente: buscó, leyó la página del paper y respondió	0,0007
5	Evidencia	`comparaciones.jsonl`, `corrida.json` y la pestaña de llamadas con tokens, segundos y dólares	0
6	Resultados	Tabla del piloto y estimaciones: piloto 0,51 USD; corrida completa 4,67 USD	0

Frases clave por paso

Paso 1

> *La memoria del WikiLLM no es una caja negra: es Markdown legible, con una cita por afirmación y una copia versionada por cada paper.*

Paso 2

> *Con 5 papers el corpus no tiene la respuesta; lo correcto es abstenerse, no inventar. Los dos lo hacen bien.*

Paso 3

> *El modelo propone; el código decide qué se guarda. Ninguna cita entra al wiki si no apunta a un fragmento real.*

Paso 4

> *Misma respuesta, misma fuente, mismo modelo. En una pregunta puntual el RAG es más barato; la apuesta del wiki está en síntesis y actualización, y eso lo mide el banco, no una pregunta.*

Paso 6

> *Es un piloto de 17 preguntas: demuestra que la cadena de evaluación funciona; no es un resultado del informe.*

Lo que muestra el ensayo del paso 4

Sistema	Llamadas	Tokens de entrada	Tokens de contexto	Segundos	USD
RAG simple	1	1.503	1.389	1,1	0,000239
WikiLLM	3	3.294	1.831	2,8	0,000421

Lectura: el RAG usó 2,2× menos tokens de entrada, 3× menos llamadas, 1,8× menos dólares y 2,5× menos tiempo. Los dos acertaron y citaron el mismo fragmento.

4. Resultados del piloto y cómo contarlos

Piloto `piloto-1`: 17 preguntas (T1, T4 y T5), pasos 0 y 5, 68 respuestas, `deepseek-flash`, temperatura 0. Marcado como `valida_para_informe: false`: usa borradores y tiene un n pequeño. Solo 6 de las 17 preguntas son respondibles en el paso 5.

Métrica (paso 5)	Mejor si es	RAG	WikiLLM	Gana
Cobertura (aciertos entre respondibles)	mayor	0,33 (2/6)	**0,83 (5/6)**	WikiLLM
Abstención correcta	mayor	1,00	1,00	empate
Fundamentación de las citas	mayor	1,00	1,00	empate
Contexto medio (tokens)	menor	**1.217**	3.544	RAG
Tokens por acierto	menor	**10.341**	12.049	RAG
Costo total (USD)	menor	**0,0027**	0,0123	RAG

Cómo contarlo con honestidad

- El WikiLLM acierta más preguntas, pero con $n = 6$ eso **no es evidencia**: solo muestra que la cadena arnés → métricas funciona.
- El costo por acierto es parecido: el wiki gasta más por pregunta, pero acierta más; por eso la diferencia en tokens por acierto es pequeña (16 %).
- La métrica que define la hipótesis (respuestas obsoletas en T4) todavía no tiene casos: hace falta la corrida completa.

Hallazgo útil para la discusión: premisa falsa

En la primera tanda del piloto, ante la T5 AT5-T5-12 (un método inexistente, «HyperAddress»), el WikiLLM **no se abstuvo**: respondió «The corpus does not mention "HyperAddress"; the relevant method is HippoRAG 2…» con una cita válida. Sustituyó la premisa por otra del corpus. En la segunda tanda sí se abstuvo. Con temperatura 0 el proveedor no es determinista, así que una sola corrida por pregunta tiene ruido. Es un buen ejemplo para justificar repeticiones y la métrica de abstención.

Costo previsto. Corrida completa: 60 preguntas × 11 pasos × 2 sistemas = 1.320 respuestas y 50 ingestas; cota alta de **4,67 USD** (1,50 de ingesta y 3,17 de preguntas). La extrapolación del piloto la sitúa entre 1 y 2,5 USD.

5. Preguntas probables del jurado

¿Por qué el mismo modelo?
Es la única variable: cambia la arquitectura de la memoria, no el modelo (constitución, regla 1). Fue un pedido del tutor.

¿No es más barato el RAG?
En preguntas puntuales sí, y lo mostramos en vivo. El experimento mide si el wiki compensa en síntesis entre papers (T2, T3) y en actualización de cifras (T4), y reporta el costo de todas formas.

¿Por qué el wiki tiene más llamadas que el RAG?
Es parte de la arquitectura: el agente decide qué buscar y leer. No se iguala; se mide con tokens, latencia y costo por respuesta.

¿Cómo saben que una cita es real?
Toda cita debe resolver a un `span_id` del Gold con la misma huella y de un paper ya ingerido en ese paso. En la ingesta, el código quita las que no resuelven antes de guardar la página.

¿Quién juzga la correctitud?
Primero una capa determinista con claves normalizadas. Lo indeterminado lo juzga otro modelo, `gemini-3.1-flash-lite`, distinto del que responde y de los que redactaron las preguntas (decisión 0003).

¿El modelo no sabe ya las respuestas?
Casi todo el corpus es de 2026, posterior al corte de entrenamiento. Las dos semillas anteriores (HippoRAG y HippoRAG 2) se marcan y se reportan aparte, y se mide un control «libro cerrado».

¿Por qué el piloto «no vale»?
n pequeño y preguntas en borrador, sin el acuerdo entre autores. Las cifras del informe saldrán solo de corridas guardadas en `results/`.

¿Y si un paper dice algo falso?
La respuesta correcta sale del corpus, no del mundo: el sistema debe repetirlo con su fuente (regla

3).

¿Qué falta?
Cerrar la verificación de la mitad B del banco y el acuerdo entre autores (kappa de Cohen), calibrar el juez, ejecutar la corrida completa, el análisis y el informe.

6. Plan B y cierre

- **Si falla la conexión o el proveedor**: en Preguntar e Ingerir, el botón **Ejecuciones guardadas** reproduce un ensayo sin llamar al modelo. El panel muestra «REPRODUCCIÓN… No es una ejecución en vivo»; hay que decirlo en voz alta.
- Los pasos 1, 5 y 6 nunca llaman al modelo.
- Si la ingesta falla a mitad: **Sistema** → **Reiniciar el wiki** y usar la reproducción.
- **Al terminar**: Reiniciar el wiki deja todo en el paso 5 y aparta la evidencia en `results/demo/ensayos/`.

Pendientes fuera de la demo

- Los cambios de hoy (panel, verificación, métricas regeneradas y bitácora) **no tienen commit**: el panel es código de Juan David; conviene que lo revise.
- Guion detallado: `docs/presentacion/guion-demo.md`.